

Sprint 2

Integrantes:

Álvaro Lima Santos - 2021067062

Beatriz Nogueira Alvares - 2023001603

Lucas Dolabella de Castro Lopes - 2023001549

Sarah Menks Sperber - 2023001824

Vanessa Nascimento Silva - 2023001816

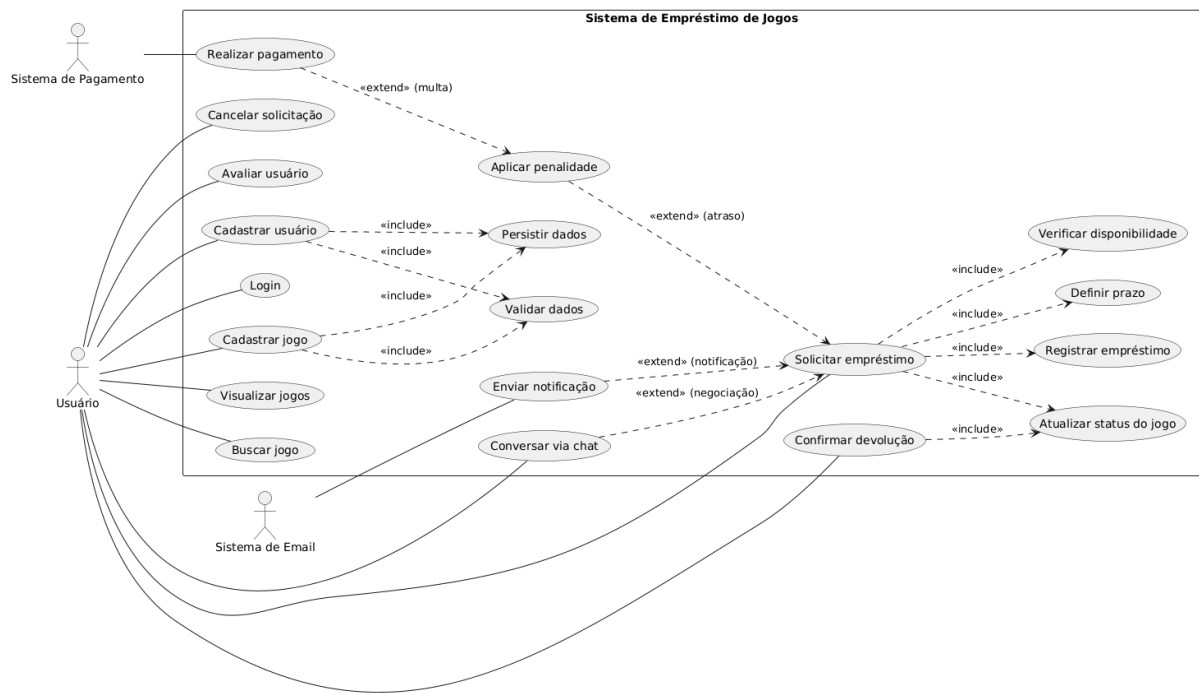
1. Diagramas de Casos de Uso

1.1. Descrição do diagrama de Casos de Uso

O diagrama de casos de uso apresenta as principais funcionalidades do sistema de empréstimo de jogos, bem como sua interação com os atores externos. O ator principal é o usuário, responsável por realizar operações como cadastro, autenticação, gerenciamento de jogos e solicitação de empréstimos. Além disso, o sistema interage com atores externos, como o sistema de e-mail, utilizado para envio de notificações, e o sistema de pagamento, responsável pelo processamento de multas.

Os casos de uso foram organizados de forma hierárquica, com a utilização de relações de generalização para agrupar funcionalidades semelhantes, como o gerenciamento de conta, jogos e empréstimos. Também foram utilizadas relações do tipo <<include>> para representar comportamentos obrigatórios reutilizáveis, como validação de dados, verificação de disponibilidade e atualização de status. Já as relações <<extend>> representam funcionalidades opcionais ou condicionais, como aplicação de penalidades, envio de notificações e comunicação via chat durante o processo de empréstimo.

Dessa forma, o diagrama fornece uma visão geral das funcionalidades do sistema, evidenciando tanto as operações principais quanto os comportamentos auxiliares e condicionais.



2. Cenários de casos de uso

Cenário 1: Solicitar Empréstimo

Atores: Usuário

Pré-condição:

- Usuário autenticado
- Jogo cadastrado
- Jogo disponível

Fluxo normal:

1. Usuário seleciona um jogo;
2. Sistema verifica disponibilidade;
3. Usuário define prazo de empréstimo;
4. Sistema valida dados;
5. Sistema registra o empréstimo;
6. Sistema atualiza status do jogo;
7. Sistema envia notificação ao dono;
8. Solicitação é confirmada.

Fluxos alternativos:

1. 2A: Jogo indisponível → operação cancelada

2. 4A: Dados inválidos → sistema solicita correção
3. 7A: Falha ao enviar notificação → processo continua sem notificação

Pós-condição:

- Empréstimo registrado
- Jogo marcado como indisponível

Cenário 2: Aplicar Penalidade

Atores: Sistema

Pré-condição:

- Empréstimo vencido
- Devolução não realizada

Fluxo normal:

1. Sistema identifica atraso;
2. Sistema aplica penalidade;
3. Sistema bloqueia usuário;
4. Sistema notifica usuário.

Fluxos alternativos:

- 2A: Sistema não consegue aplicar penalidade → registrar erro

Pós-condição:

- Usuário bloqueado
- Penalidade registrada

Cenário 3: Confirmar Devolução

Atores: Usuário (Dono do jogo)

Pré-condição:

- Usuário autenticado.
- Empréstimo ativo associado ao usuário.
- Prazo de devolução vencido

Fluxo normal:

1. Usuário acessa notificação de confirmação de devolução;
2. Sistema exibe detalhes do empréstimo (jogo, solicitante, prazo);
3. Usuário confirma que o jogo foi devolvido;
4. Sistema registra a devolução;
5. Sistema atualiza status do jogo para "Disponível";

6. Sistema libera o solicitante de qualquer pendência;
7. Sistema envia notificação ao solicitante confirmando a devolução.

Fluxos alternativos:

- 3A: Usuário informa que o jogo não foi devolvido → sistema registra a não devolução, gera multa automaticamente e bloqueia conta do solicitante

Pós-condição:

- Empréstimo finalizado
- Jogo disponível para novos empréstimos (se devolvido)
- Aviso para solicitante e multa (se não devolvido)

Cenário 4: Realizar Pagamento de Multa

Atores: Usuário (Solicitante bloqueado)

Pré-condição:

- Usuário autenticado
- Usuário com acesso bloqueado
- Multa pendente registrada no sistema

Fluxo normal:

1. Usuário acessa área de pendências;
2. Sistema exibe multa pendente com valor e motivo;
3. Sistema encaminha para tela de pagamento;
4. Usuário seleciona método de pagamento;
5. Usuário realiza o pagamento;
6. Sistema registra pagamento da multa;
7. Sistema atualiza status da pendência para "Quitada";
8. Sistema desbloqueia acesso do usuário a novos empréstimos;
9. Sistema envia comprovante de quitação por e-mail.

Fluxos alternativos:

- 3A: Usuário desiste do pagamento → Operação cancelada, bloqueio mantido.
- 5A: Pagamento recusado → Multa permanece pendente e bloqueio é mantido.
- 6A: Timeout na confirmação do pagamento → Sistema registra pagamento com status "Em verificação" e bloqueio é mantido até confirmação do usuário

Pós-condição:

- Multa quitada
- Acesso do usuário desbloqueado

- Comprovante enviado

Cenário 5: Conversar via Chat

Atores: Usuário (Remetente e Destinatário)

Pré-condição:

- Ambos os usuários autenticados
- Jogo visualizado pelo remetente
- Destinatário é o dono do jogo (ou solicitante, em conversa já existente)

Fluxo normal:

1. Usuário seleciona opção "Conversar" no anúncio do jogo;
2. Sistema verifica se já existe conversa ativa entre os usuários para este jogo;
3. Sistema abre interface de chat;
4. Usuário digita e envia mensagem;
5. Sistema armazena mensagem;
6. Sistema exibe mensagem no chat do destinatário em tempo real;
7. Sistema envia notificação ao destinatário sobre nova mensagem.

Fluxos alternativos:

- 2A: Já existe conversa ativa → Sistema carrega histórico da conversa e continua.
- 4A: Usuário tenta enviar mensagem vazia → Sistema impede envio.
- 6A: Destinatário não está online → Destinatário recebe notificação.

Pós-condição:

- Mensagem armazenada no histórico
- Destinatário notificado
- Canal de comunicação estabelecido para negociação

3. Projeto arquitetural

3.1. Padrão Arquitetural Escolhido

O sistema proposto adota o padrão de Arquitetura em Camadas (Layered Architecture). Nesse modelo, o sistema é organizado em um conjunto de camadas hierárquicas, nas quais cada camada desempenha um papel específico. Cada camada fornece serviços para a camada imediatamente superior e, ao mesmo tempo, consome serviços da

camada inferior, promovendo uma separação clara de responsabilidades e facilitando a organização do sistema.

3.2. Estrutura das Camadas

A arquitetura do sistema foi dividida em cinco camadas principais:

1. Camada de Apresentação (Presentation)

Responsável pela interação com o usuário, essa camada corresponde ao frontend da aplicação. Suas principais funções são a entrada de dados e a exibição das informações processadas pelo sistema, garantindo uma interface clara e intuitiva.

2. Camada de Aplicação (Application)

Essa camada atua como intermediária entre a interface e as regras de negócio. É responsável por controlar o fluxo do sistema, orquestrando as requisições e gerenciando a comunicação entre o frontend e os serviços da camada de negócio.

3. Camada de Negócio (Business)

Contém as principais regras e lógicas do sistema. Essa camada foi organizada em diferentes subsistemas, responsáveis por funcionalidades específicas, como autenticação de usuários, gerenciamento de jogos, controle de empréstimos, aplicação de penalidades e comunicação via chat.

4. Camada de Dados (Data)

Responsável pela persistência das informações, essa camada gerencia o acesso ao banco de dados, que é centralizado e compartilhado entre os demais componentes do sistema.

5. Camada de Sistemas Externos (External Systems)

Essa camada representa a integração do sistema com serviços externos. Inclui, por exemplo, serviços de envio de e-mails e processamento de pagamentos. Esses sistemas são utilizados para funcionalidades específicas, como notificações automáticas e quitação de penalidades, e se comunicam com a camada de negócio por meio de interfaces bem definidas.

3.3. Justificativa da Escolha

A adoção da arquitetura em camadas foi motivada por diversos fatores. Primeiramente, esse padrão favorece o desenvolvimento incremental, permitindo a evolução do sistema por partes bem definidas. Além disso, contribui para uma manutenção mais simples, uma vez que alterações em uma camada tendem a impactar minimamente as demais.

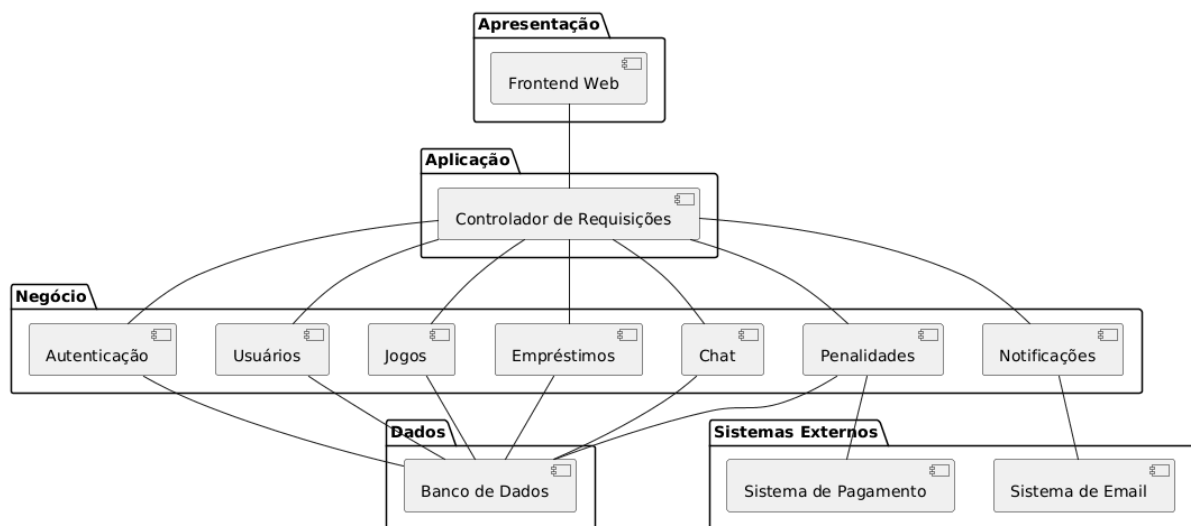
Outro ponto relevante é o isolamento das regras de negócio, concentradas em uma camada específica, o que melhora a organização e a reutilização do código. A arquitetura também facilita a substituição de componentes, desde que as interfaces entre as camadas sejam mantidas estáveis.

Do ponto de vista dos requisitos não funcionais, essa abordagem contribui significativamente para a qualidade do sistema. A segurança é reforçada ao manter regras críticas nas camadas internas, o desempenho é favorecido pela separação de responsabilidades, e a manutenção é aprimorada devido ao baixo acoplamento entre os componentes.

3.4. Decisões Arquiteturais

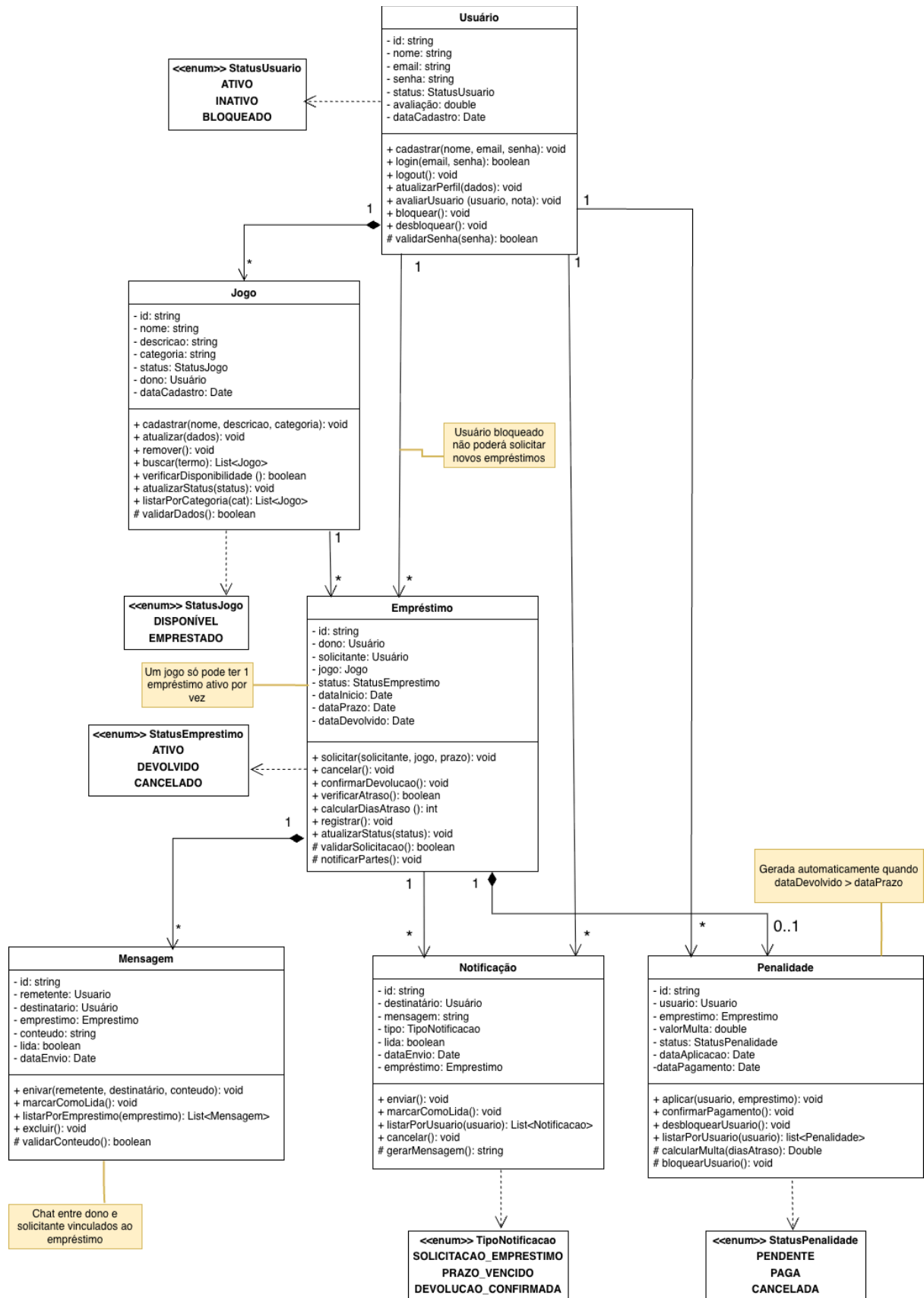
Com base nos requisitos do sistema e nas diretrizes do projeto arquitetural, foram definidas as seguintes decisões:

- O sistema será implementado como uma aplicação centralizada, seguindo um modelo monolítico modular;
- A comunicação entre os componentes será realizada por meio de uma API interna, baseada em padrão REST;
- Será utilizado um banco de dados único e centralizado, caracterizando um repositório compartilhado entre os subsistemas;
- O sistema realizará integração com serviços externos, como envio de e-mails e processamento de pagamentos, necessários para funcionalidades de notificação e penalidades.



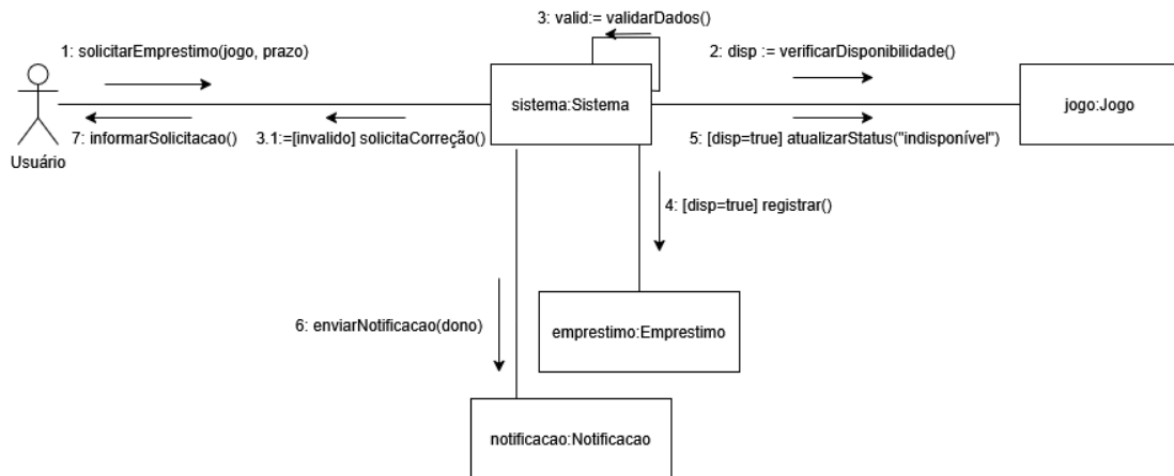
4. Projeto detalhado do sistema

4.1 Diagrama de Classes

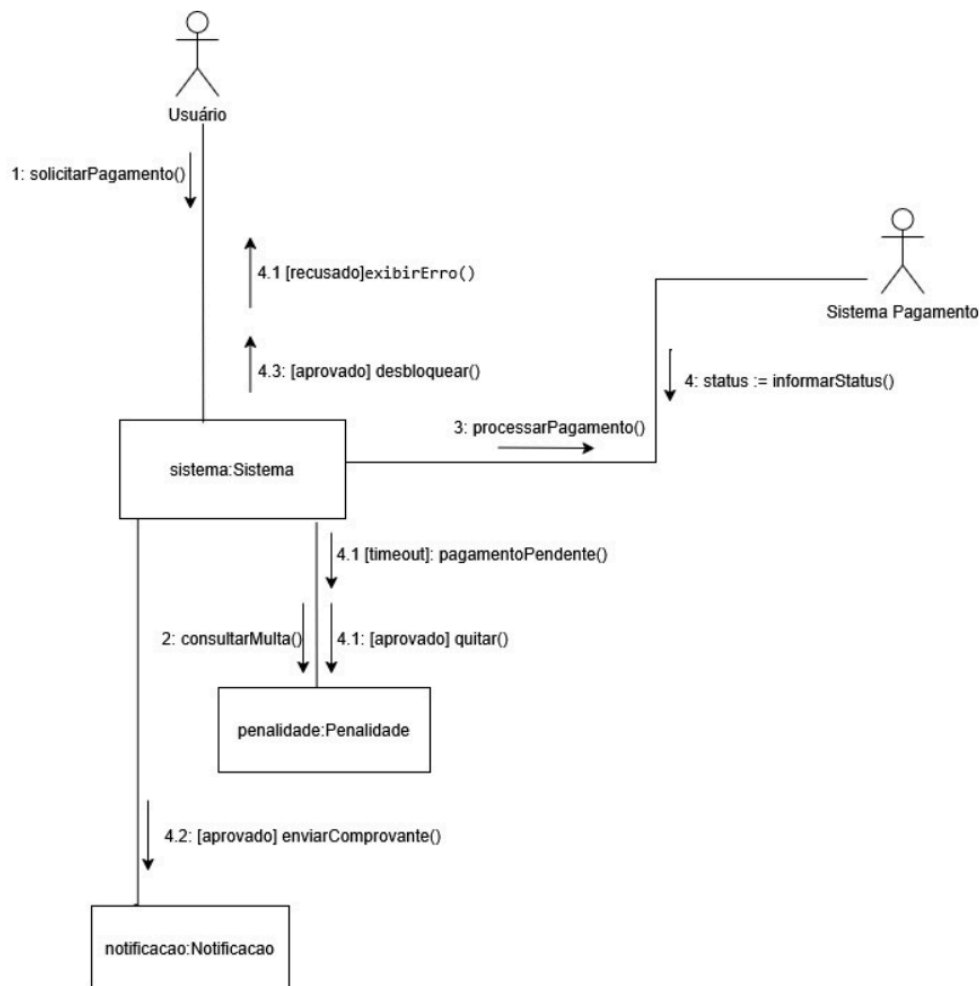


4.2 Diagramas de Comunicação

- Solicitar empréstimo

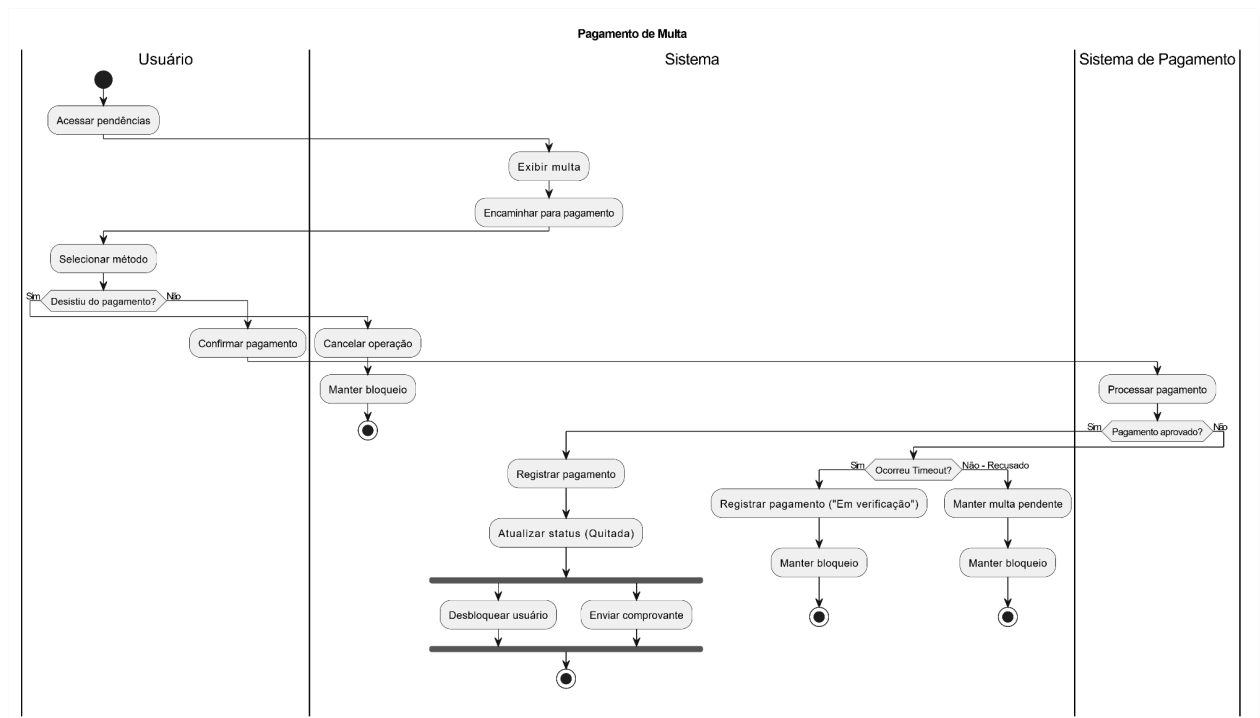


- Realizar pagamento de multa



4.3. Diagramas de Atividade

- Realizar pagamento de multa



- Solicitar empréstimo



5. Conclusão

A Sprint 2 permitiu a definição da arquitetura e da modelagem detalhada do sistema. A partir dos requisitos levantados na Sprint 1, foram elaborados os diagramas de casos de uso com a descrição dos principais cenários, contemplando os fluxos normais e alternativos das funcionalidades centrais do sistema, como solicitação de empréstimo, confirmação de devolução, aplicação de penalidade, pagamento de multa e comunicação via chat.

Foi definido o projeto arquitetural do sistema, adotando a Arquitetura em Camadas como padrão, organizada em cinco camadas: Apresentação, Aplicação, Negócio, Dados e Sistemas Externos. Essa escolha favorece a separação de responsabilidades, facilita a manutenção e suporta o desenvolvimento incremental adotado pela equipe.

No projeto detalhado, foi elaborado o Diagrama de Classes com as seis entidades principais do sistema: Usuário, Jogo, Empréstimo, Mensagem, Notificação e Penalidade. Além disso, foram incluídos atributos, métodos, visibilidades e relacionamentos. Complementando o projeto detalhado, foram desenvolvidos Diagramas de Comunicação e Diagramas de Atividade para os cenários de solicitação de empréstimo e pagamento de multa, evidenciando o fluxo de interações entre os objetos e as atividades realizadas pelos atores do sistema.

Os artefatos produzidos nesta sprint estabelecem uma base técnica sólida para o início da implementação na Sprint 3, onde as funcionalidades modeladas serão desenvolvidas e validadas por meio de testes.

6. Bibliografia

FIGUEIREDO, Eduardo. *Engenharia de Software I*. Slides de aula. Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, 2026.

SOMMERVILLE, Ian. *Engenharia de software*. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

G. Booch, J. Rumbaugh, I. Jacobson. *UML, Guia do Usuário*. 2ª Ed., Editora Campus, 2005.

M. Fowler. *UML Essencial*, 2a Edição. Bookmann, 2000.